Bulk Category Editor

Full Audit Report

Tile Station Admin · Fork session · 3 Aug 2026

Bulk Category Editor — Full Audit Report

Audit date: 3 Aug 2026 — Fork session **Files audited**: `SupplierProducts.js` (16,409 lines), `BulkCategoryEditorSections.jsx` (2,222 lines), `BulkCategoryEditorModal.jsx` (715 lines, **DEAD CODE**), the 15 sibling components under `/components/supplier-products/`, and the 3 backend endpoints (`bulk-update-unified`, `bulk-description`, `bulk-tier-update`).

Status: READ-ONLY. Zero code changes made.

TL;DR — the three root causes of your pain

- **THREE save paths that behave subtly differently.** "Save Now" (dry-run + bulk API), "Force Save" (per-product loop + progress), and "Quick Save" (per-attribute inline). Same button label style, different guarantees, different post-save behaviour. Users can't tell which one is authoritative.
- **A fan-out to 5 Mongo collections + AI/collection cache**, done inside the request handler with implicit rules. When something "doesn't sync", it's because one of those collections was skipped (e.g. `tiles` is only auto-published when `show_on_website=true` AND `sub_categories` was set).
- **Two overlapping description generation systems** with different endpoints, different result shapes, and different persistence targets, both living inside the same modal panel.

Everything else is symptoms of these three.

---

1. Save & Sync — the biggest headache

Symptom (as reported)

> ""sometimes they don't sync … same work takes 3 times to do the same but still doesn't work/save properly""

Root cause

There are **THREE completely separate save code paths**, all in `SupplierProducts.js`:

Path	Function	Entry point	API called	Batching
A. Save Now	`handleBulkCategoryUpdate` (line ~4270)	Green "Save Now to N Products" → Dry Run → confirm	`POST /api/supplier-sync/products/bulk-update-unified` with ALL product IDs in **one payload**	1 request for all products
B. Force Save	`handleForceSave` (line 4768)	Amber "Force Save" button (skips dry-run)	Same endpoint, but **one request per product** in a loop	N requests, progress bar
C. Quick Save	`handleQuickSave` (line 5254)	Cloud icon next to a single attribute inside a section	Same endpoint, batches of 10 in parallel	N/10 batches

Consequences that match your "3 tries" symptom:

- Path A uses one payload; if the backend has any subtle validation error mid-batch, the whole payload may partially succeed silently. There is `runSaveVerification` (line 4707) that re-fetches the products and compares, but its result is only shown in a small banner — users don't always notice.
- Path B is more resilient (per-product), but doesn't go through Dry Run — so if your selections had a bad value (e.g. wrong finish), you get 25/25 "success" but the wrong thing saved.
- Path C only saves the ONE attribute — but the user's mental model is "I've changed 5 things, hit that cloud icon, and now everything's saved". It isn't. The other 4 attributes are still dirty. This is a real UX foot-gun.

The sync fan-out inside `bulk_update_unified`

Every save touches these Mongo collections in this order (see `supplier_sync.py:11001`):

```
sync_staging → supplier_products → products → tiles (customer-facing) →
collection_settings (series description)
```

Silent skip rules (this is why "sometimes it doesn't sync"):

- `tiles` **(storefront) is only auto-populated** when the request has BOTH `show_on_website: true` in the updates AND a category change (`sub_categories` or `main_category`). Otherwise the product stays on the admin side but never appears on the website. See lines 11288–11296.
- `collection_settings.custom_description` is only written from the frontend, after a template save, and only in `handleForceSave` + `handleBulkCategoryUpdate` — **NOT** in `handleQuickSave`. So a Quick Save on description does not push it into the customer-facing collection page description.
- **Attribute sync between** `filter_*` **and** `spec_*`: the frontend has custom code that mirrors `spec_finish` to `filter_finish` in one direction only. If you set `filter_finish` first, you can end up with `finish` on the top-level doc mismatched from what shows in filters.
- `array_to_scalar_sync` on the backend (line 11151) auto-copies `colors[0]` → `color`, `materials[0]` → `material`, `sub_categories[0]` → `category`. Fine in isolation, but bad news if a user has _also_ manually set the scalar to something different in the same request — the

auto-sync silently overwrites.

- **Publishing to `tiles`** (customer-facing) builds a brand-new document from `supplier_products` — if `our_product_name` was changed AFTER a previous publish, the storefront may hold the old name in `tiles.name` until re-published.

Verdict

You've got **3 save buttons**, **5 collections**, and **6 implicit sync rules** — nobody can hold that in their head. That's why saves feel unreliable.

Proposed fix (P0)

Collapse to ONE Save button, with a single robust write pipeline:

- Rename the primary green button to just **"Save"** (drop the "Now to N products" jargon).
- Behind the button, ALWAYS go through:

- **Client-side dry run** (already built — `DryRunPreview`) with a "just do it" toggle for advanced users.
- **Backend transactional save** in `POST /api/supplier-sync/products/bulk-update-unified` (already exists) — but wrap the fan-out in ONE MongoDB transaction (or a compensating rollback log) so partial saves either fully succeed or fully fail.
- **Backend returns a per-product result map**: `{ sku1: 'ok', sku2: 'ok', sku3: {error: 'invalid slip_rating'} }` — the frontend shows exactly which products failed inline.
- **Automatic verification** (already built — `runSaveVerification`) becomes required, not optional. If DB doesn't reflect the intended changes within 2s, we flag it prominently and show the mismatched fields.

- **Delete "Force Save"** — its only useful behaviour (per-product resilience) is subsumed by returning the per-product result map above.
- **Rename "Quick Save"** to a clearly scoped icon: a single "Save just this attribute" icon that says on the tooltip: **"Saves this one attribute to all selected products and clears the pending change from the modal. Does not save your other pending edits."** — and add a big "See 4 other unsaved edits" banner at the top of the modal so users know what they haven't saved.

---

2. Description generation — the second headache

Symptom

> ""description generation … multiple different ways … really pain""

Root cause

Four different generation entry points, four different endpoints, three different persistence behaviours, all inside the same modal panel:

ol	Function	Endpoint	Where it writes		
ription **tor** (blue te" — legacy, noved nt says)	~~removed 24 May 2026~~	—	—		
tion **te** textarea + ith template" handleBulkCat odate	`handleBulkGenerate Description('generate' \	'brief' \	'long')`	`POST /api/products/generate-description`	Per-product `description` save
/ Longer / v (appears mplate exists)	`handleBulkGenerate Description('shorter' \	'longer' \	'regenerate')`	Same endpoint, different `mode`	Same
Series tion — 3 Brief / d / Detailed	`handleGenerateSeriesDescription(length)`	`POST /api/products/generate-series-description`	Per-product `description` + `collection_settings.custom_description`		
Generate All Descriptions	`handleOpenBatchSeriesModal` → `POST /api/products/generate-batch-series-descriptions`	Different endpoint	Same as above		
egeneration	Opens another modal	—	Scheduled worker		

The user has to know:

- Whether to generate PER PRODUCT (template with `{color}` placeholders) or FOR THE SERIES (one unified paragraph).
- Whether their generation writes to the customer-facing collection page description or not (it does for series, doesn't for template — completely unintuitive).
- Whether the "SEO keywords" field they're typing in applies to the current generation or the next one.

Proposed fix (P0)

Reduce to ONE description panel with two clear modes:

```
Description
■ Per-product (with placeholders like {color}, {size})
● Unified series (one description for the whole collection)

[Textarea — currently empty]

[ Generate ] Length: Brief Standard Detailed
[ Shorter ] [ Longer ] [ New variation ] ← only visible after generation
```

- **One toggle** — per-product vs unified series. This determines which endpoint gets called.

- **One "Generate" button** with a length selector chip row.
- **Shorter/Longer/New** only after a generation exists.
- **Kill the separate "Quick Generate All" button** — it belongs inside "Unified series" mode with a "for all selected series" toggle.
- **Kill "Auto-Regeneration Settings"** as a modal-inside-a-modal — move it to a dedicated Settings page since it's a background job.

Backend consolidation (P1, not P0):

- One endpoint: `POST /api/products/generate-description-v2` with `{ scope: 'per-product' | 'unified', length: 'brief' | 'standard' | 'detailed', products: [...], seo_keywords: '', mode?: 'shorter' | 'longer' | 'new' }`.
- Deprecate `/generate-description`, `/generate-series-description`, `/generate-batch-series-descriptions` (keep them alive for 2 weeks to avoid breaking anything, then delete).

---

3. Fragile structure — the third headache

Symptom

> ""too many features depending on each other for them to work properly""

Root cause

`SupplierProducts.js` **has 164** `useState` **calls** and is 16,409 lines long. State examples that overlap and depend on each other:

- `bulkCategorySelections` — the giant "what did the user select in the modal" object (~40+ different keys)
- `perAttributeScopes` — Map keyed by `attrKey__value` → Set of `productKey` (per-value scoping)
- `perProductAssignments` — nested Map: `attrKey` → `productKey` → value
- `sectionProductScopes` — `{ categories: Set, filters: Set, specifications: Set }`
- `fieldsToClear` — Map of field → array of values to remove
- `bulkDescriptionSettings` — description-related object
- `bulkSaleSettings`, `bulkTilesPerBoxSettings`, `bulkThicknessFilter`, `pricingSizeFilter`, `tierPricingConfig`, `tierProductScope`, …

Each save path reads a different subset of these. **Force Save** reads slightly different state from **Save Now** (Force Save checks `hasFilterFinishScope` etc. to gate scalars, Save Now doesn't). Meaning: **the two paths can produce different results from the same UI state.**

Additionally:

- The **legacy** `BulkCategoryEditorModal.jsx` (715 lines) is **NOT imported anywhere**. It's dead code that just adds confusion when grepping.
- The `Sections` component takes **26 props** — most of them setters passed straight from the parent. This is a strong sign the modal's state should live in a reducer/store, not scattered on `SupplierProducts.js`.

- The `restore draft` mechanism (`saveDraftToServer` / `clearServerDraft`) can silently restore a stale draft that no longer matches the currently-selected products. When you clicked "Restore" on the yellow "Unsaved draft found from 29/05/2026" banner in your screenshot, that's saved state from **~2 months ago** — huge foot-gun.

Proposed fix (P1)

- **Delete** `BulkCategoryEditorModal.jsx` — dead code.
- **Extract a** `useBulkEditorState` **hook** in a new file — pulls all 40+ bulk-related states out of the parent into one place. The parent becomes ~11k lines instead of 16k. Two weeks of gain in maintainability.
- **Consolidate the "Force Save" and "Save Now" paths** into a single function `saveBulkEdits({ mode: 'batch' | 'per-product' })`. Same code path, same state reads. Eliminates the "produces different results" class of bugs by construction.
- **Add a draft freshness check**: draft-restore banner should ONLY show if the draft's `selectedProducts` matches the currently-selected products (or offer a "reset selection" option). Otherwise silently discard the draft.

---

4. Pricing / Tier discounts — asked to check

Looked at:

- `openTierPricingModal` (line 967)
- `handleTierPricingSave` (line 2380+) — one big `/products/bulk-tier-update` endpoint that overrides `tier_thresholds`, `tier_discounts`, `trade_discount`, `credit_back_rate`, `disabled` — with an optional `tierProductScope`.
- Global tiers settings (line 2424) — `/tiles/pricing/tiers` PUT
- The `DiscountCalculatorPreview` panel showing what the customer sees.
- `pricingSizeFilter` + `bulkThicknessFilter` — used in `handleForceSave` and pricing save to gate which products a price change applies to.

Findings:

- **Two independent scope selectors** — `tierProductScope` (from TierPricingModal), `pricingSizeFilter` (a chip filter on the parent), and `bulkThicknessFilter` (another chip). Save Now respects `bulkThicknessFilter` (line 4779), pricing save respects `pricingSizeFilter`, but a user can have BOTH set and the priority isn't obvious. Not a bug per se, but very confusing UX.
- `price` **gets duplicated into** `list_price` **AND** `room_lot_price` **in one write** (line 11102). Good. But `pallet_price` is computed as `list_price * 0.9` only on tile-publish (line 11348), not on price update. Result: if you change price but don't re-publish, `pallet_price` becomes stale on the customer-facing `tiles` doc.
- `cost_price` **has no validation** — accepts any float, including 0 or negative. Combined with the tier discount logic, this can silently produce £0.00 tiles on the front-end.
- **Tier "disabled"** is a boolean stored on the product. When disabled, the frontend shows a different chip. But the toast messaging is inconsistent ("Tier discounts DISABLED for N products" vs "Saved: X% discounts for N products" — different structure) so it's hard to scan.
- `custom_quote_threshold` — a global setting (default 150m²) beyond which the customer sees "Request Quote" instead of "Add to Cart". Not obvious that changing it affects live checkout — no

confirmation step.

Proposed fix (P1)

- Add server-side validation: `cost_price >= 0`, `list_price >= 0`, `list_price >= cost_price * 0.5` (warning, not error), tier discounts monotonically increasing.
- Recompute `pallet_price` on every price update, not just on publish.
- Show a single unified "Scope" line at the top of every bulk action: "Applying to X products — filtered by size 60×60, thickness 10mm". Same UI regardless of which action (categories, pricing, tiers) is being edited.

---

5. Website Preview / Publish — asked to check

- The "Website Preview" panel at the top of the modal previews only the FIRST selected product. If you have 3 products with different names, you only see one. Minor UX issue.
- "Publish to Website" button appears when `show_on_website === false` on the first product. Applies to only the FIRST product? Or all? Reading `publishSelectedToWebsite` — it applies to all selected. But the panel labels are ambiguous.
- Auto-publish rule (backend line 11288–11296): products that don't yet exist in the `tiles` collection get auto-created **only** when the update explicitly includes `show_on_website: true` AND has a category change. Not obvious to users — many people probably think "I set a category and hit save, why isn't it on the website?"

Proposed fix (P2)

- Add a **"Publish state"** column to the selected-products list at the top: "3 selected · 2 published · 1 draft". One-click to publish all.
- Add a **banner inside the modal** — "1 of 3 selected products isn't on the website yet. Toggle 'Show on website' at the bottom to auto-publish when you save."

---

6. Manage Options — asked to check indirectly

- Modal-inside-modal (`ManageOptionsModal.jsx`, 1213 lines) — huge component in its own right.
- Adds/edits/deletes attribute values (colours, materials, etc.).
- **Doesn't refresh** the parent modal's option list after adding a new value — user has to close/reopen the Bulk Category Editor to see the new option. Real productivity killer.

Proposed fix (P2)

- Propagate a callback so `ManageOptionsModal.onValueAdded` triggers a re-fetch of `productOptions` on the parent.
- Add optimistic UI: the new value should appear in the dropdown immediately, before the parent re-fetches.

---

7. Presets / Draft Restore — asked to check

- `BulkEditPresets.jsx` (221 lines) — the "Presets" dropdown at the top. Saves the current selections under a name; can load them later.
- **No stale-check** — a preset saved 3 months ago with `slip_rating: R11` will still load that value even if `R11` is no longer in the options list. Silent data corruption risk if you don't notice.
- Draft restore (`saveDraftToServer`) — same issue as above: no timestamp check, no product-set match check. Restoring a stale draft can wipe your current work.

Proposed fix (P2)

- Validate preset values against current option lists on load; warn user about invalid keys.
- Draft restore only allowed when the current selected-products set matches the draft's selected-products set exactly.

PROPOSED WORK PLAN (in priority order)

P0 — do first, biggest bang for buck (roughly 2–3 sessions):

- **Unify saves**: one "Save" button. Delete "Force Save". Rename Quick Save icon + tooltip. Show pending-changes banner. Return per-product results.
- **Unify description generation**: one panel, two-mode toggle (per-product / unified series), one Generate + length chip row.
- **Fix draft-restore freshness check**: don't offer stale drafts.

P1 — do next (roughly 2 sessions):

- **Extract `useBulkEditorState` hook** — pull 40+ bulk states out of `SupplierProducts.js` into one file. Zero behaviour change, huge maintainability win.
- **Pricing safeguards**: server-side validation, recompute `pallet_price` on price update, single "Scope" line UX.
- **Backend consolidation**: one `generate-description-v2` endpoint.

P2 — nice to have (roughly 1 session):

- Publish state column + banner.
- Manage Options refresh callback.
- Preset staleness warnings.
- Delete dead code (`BulkCategoryEditorModal.jsx`).

What I need from you before I touch code

Please confirm — **either "do all P0 items" or pick the ones you want first**. Also flag anything above that you disagree with. In particular:

- **OK to delete "Force Save"?** I'll replace its resilience with per-product result reporting from the main save.
- **OK to consolidate description generation into ONE panel with a toggle?** You'll lose the four coloured section boxes but gain one clear flow.

- **OK to auto-discard stale drafts (>7 days old, or when selection changed)?** Or would you rather I show a "this draft is stale" warning and let you decide?
- **OK to add pricing validation** (cost_price ≥ 0, warn if list_price < cost_price × 0.5)? This might reject some historical imports if they had bad data — I'll show a soft warning first, not a hard rejection.